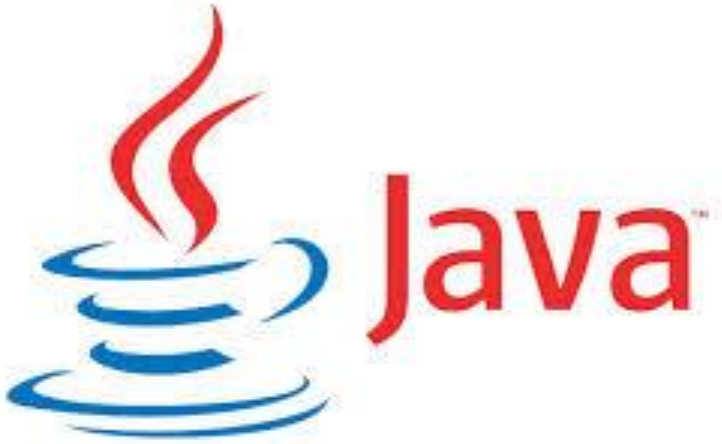


SOC2030 Application Programming in Java

Week 10 Networking & Multithreading

Dr. Andrei Dragunov

Details

- Networking Fundamentals
- The Client/Server Model
- InetAddress and URLs
- Sockets
- Stream Sockets and Datagram Sockets
- Multithreading

Introduction

Computer networking is used to send and receive messages among computers on the Network.

When a computer needs to communicate with another computer, it needs to know the other **computer's address.**

An *Internet Protocol* (IP) address uniquely identifies the computer on the Internet.

Internet Protocol

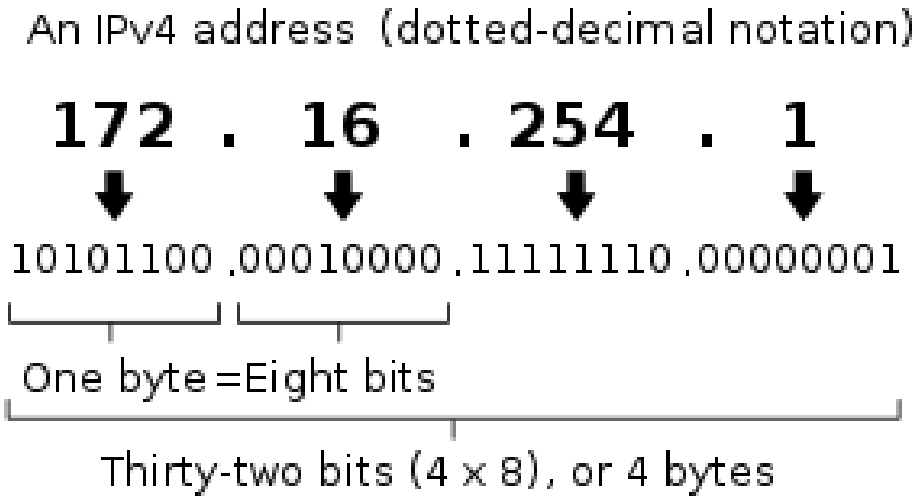
Definition:

The **Internet Protocol** (**IP**) is the principal communications protocol in the Internet protocol suite for relaying datagrams across network boundaries.

IP has the task of delivering packets from the source host to the destination host solely based on the IP addresses in the packet headers.

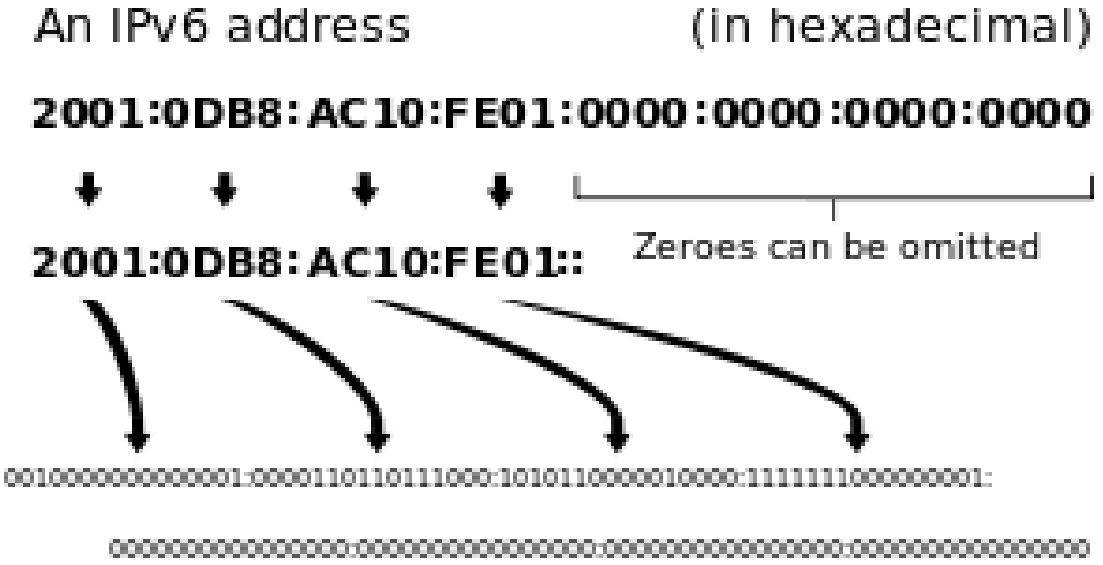
For this purpose, **IP defines packet structures that encapsulate the data to be delivered**. Also **IP defines addressing methods** that are used to label the datagram with source and destination information.

IPv4 vs IPv6 Addressing



IPv4 addresses may be represented in any notation expressing a 32-bit integer value. They are most often written in the dot-decimal notation, which consists of four octets of the address expressed individually in decimal numbers and separated by periods.

The main advantage of IPv6 over IPv4 is its larger address space. The length of an IPv6 address is 128 bits, compared with 32 bits in IPv4. The address space therefore has 2^{128} or approximately 3.4×10^{38} addresses.

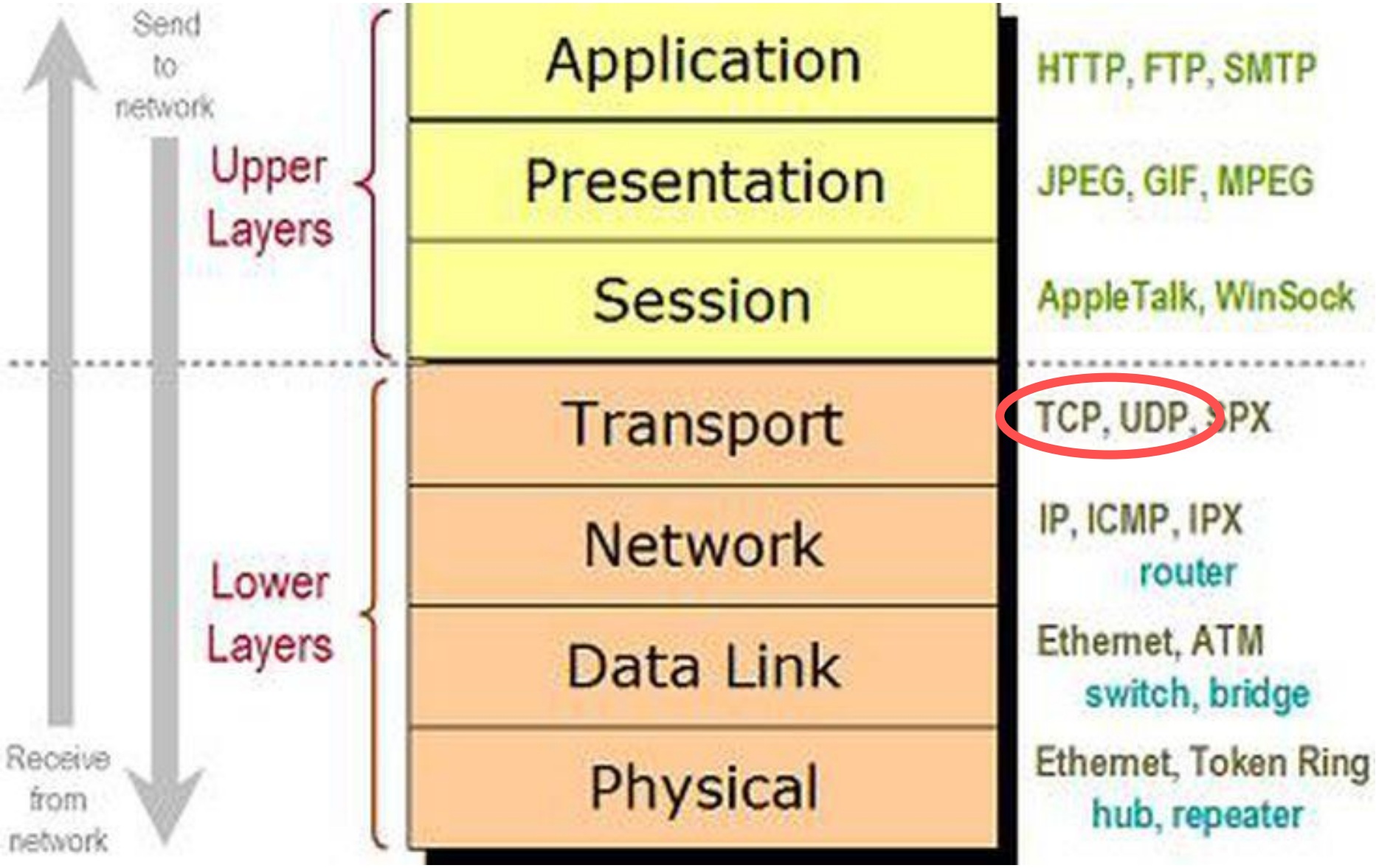


Domain Name, Domain Name Server

Since it is not easy to remember so many numbers, they are often mapped to meaningful names called *domain names*, such as eclass.inha.uz.

Special servers called *Domain Name Servers* (DNS) on the Internet translate host names into IP addresses.

Internet protocol suite



- Two higher-level protocols used in conjunction with the IP are the ***Transmission Control Protocol*** **(TCP)** and the ***User Datagram Protocol*** **(UDP)**.
- **TCP** enables two hosts to establish a connection and exchange streams of data. TCP guarantees delivery of data and also guarantees that packets will be delivered in the same order in which they were sent.
- **UDP** is a standard, low-overhead, connectionless, host-to-host protocol that is used over the IP. UDP allows an application program on one computer to send a datagram to an application program on another computer.

The client server model

Most interprocesess communication uses the *client server model*.

These terms refer to the two processes which will be communicating with each other.

- One of the two processes, **the *client***, connects to the other process, **the *server***, typically to make a request for information.
- The client needs to know of the existence of and the address of the server, but the server does not need to know the address of (or even the existence of) the client prior to the connection being established.
- The system calls for establishing a connection are somewhat different for the client and the server, but both involve the basic construct of a *socket*.
  A socket is one end of an interprocess communication channel. The two processes each establish their own socket.

Client/Server Computing

Networking is tightly integrated in Java. The Java API provides the classes for creating sockets to facilitate program communications over the Internet.

Java provides the **ServerSocket** *class for creating a server socket and the* **Socket** *class for creating a client socket. Two programs on the Internet communicate through a server socket and a client socket using I/O streams.*

Server Sockets

To establish a server, you need to:

- create a *server socket*
- attach *server socket* to a *port*, which is where the server listens for connections.

The port identifies the TCP service on the socket. Port numbers range from 0 to 65536, but port numbers 0 to 1024 are reserved for privileged services.

- Official: Port is registered with IANA* for the application.
- Unofficial: Port is not registered with IANA* for the application.
- Multiple use: Multiple applications are known to use this port.

*Internet Assigned Numbers Authority

Server Sockets

The following statement creates a server socket **serverSocket**:

```
ServerSocket serverSocket = new ServerSocket(port);
```

After a server socket is created, the server can use the following statement to listen for connections:

```
Socket socket = serverSocket.accept();
```

This statement waits until a client connects to the server socket.

Client Sockets

- The client issues the following statement to request a connection to a server:

```
Socket socket = new Socket(serverName, port);
```

- This statement opens a socket so that the client program can communicate with the server. ***serverName*** is the server's Internet host name or IP address.
- Alternatively, you can use the domain name to create a socket.

Data Transmission through Sockets

After the server accepts the connection, communication between the server and the client is conducted in the same way as for I/O streams.

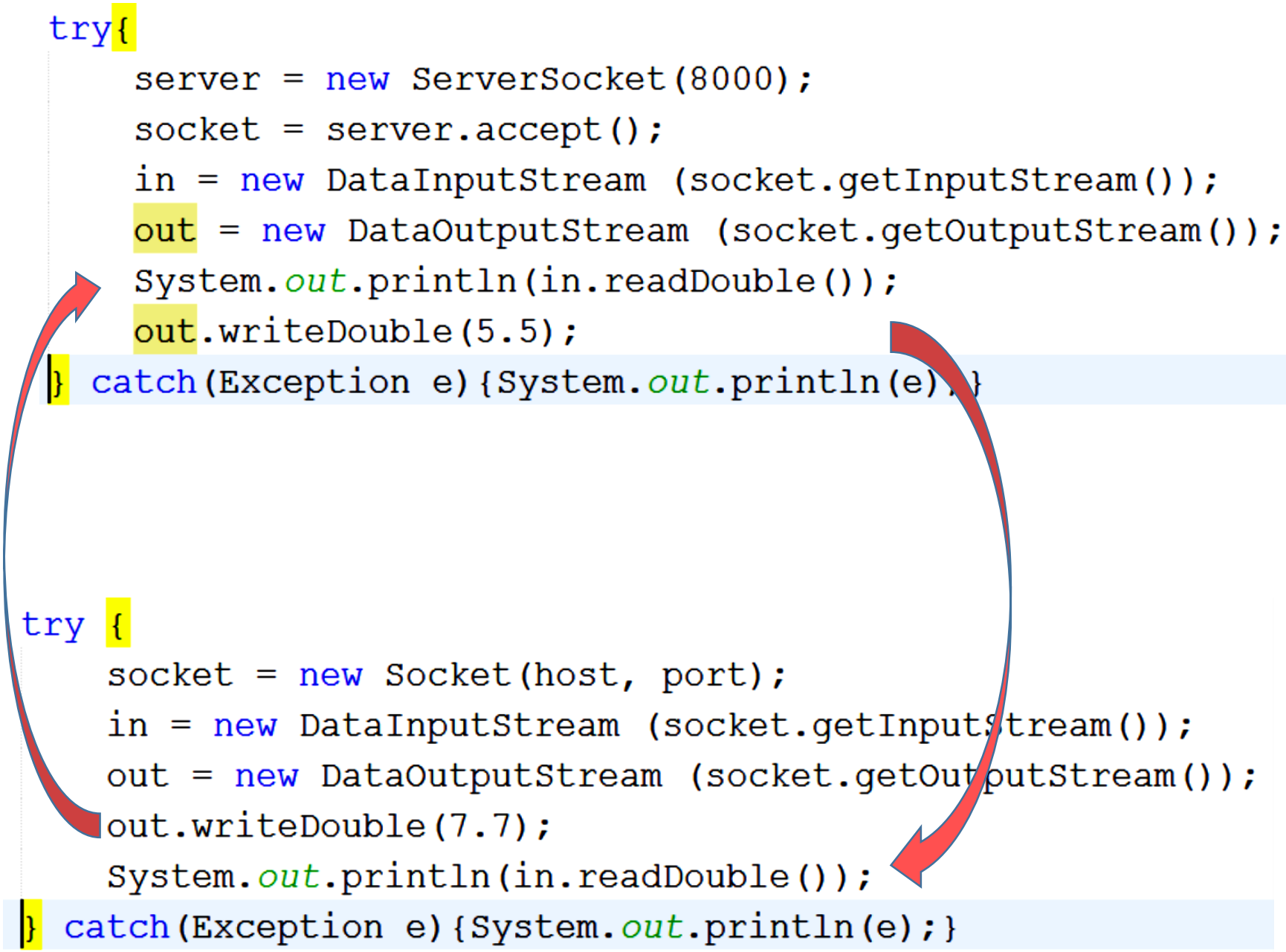
- To get an input stream and an output stream, use the **getInputStream()** and **getOutputStream()** methods on a socket object.
- For example, the following statements create an **InputStream** stream called **input** and an **OutputStream** stream called **output** from a socket:

```
InputStream input = socket.getInputStream();
OutputStream output = socket.getOutputStream();
```

The InputStream and OutputStream

The **InputStream** and **OutputStream** streams are used to read or write bytes. You can use **DataInputStream**, **DataOutputStream**, **BufferedReader**, and **PrintWriter** to wrap on the **InputStream** and **OutputStream** to read or write data, such as **int**, **double**, or **String**. The following statements, for instance, create the **DataInputStream** stream **input** and the **DataOutput** stream **output** to read and write primitive data values:

```
DataInputStream input = new DataInputStream
  (socket.getInputStream());
DataOutputStream output = new DataOutputStream
  (socket.getOutputStream());
```


try{
server = new ServerSocket(8000);
socket = server.accept();
in = new DataInputStream (socket.getInputStream());
out = new DataOutputStream (socket.getOutputStream());
System.out.println(in.readDouble());
out.writeDouble(5.5);
} catch(Exception e){System.out.println(e);}
try {
socket = new Socket(host, port);
in = new DataInputStream (socket.getInputStream());
out = new DataOutputStream (socket.getOutputStream());
out.writeDouble(7.7);
System.out.println(in.readDouble());
} catch(Exception e){System.out.println(e);}

Serving Multiple Clients

A server can serve multiple clients. The connection to each client is handled by one thread.

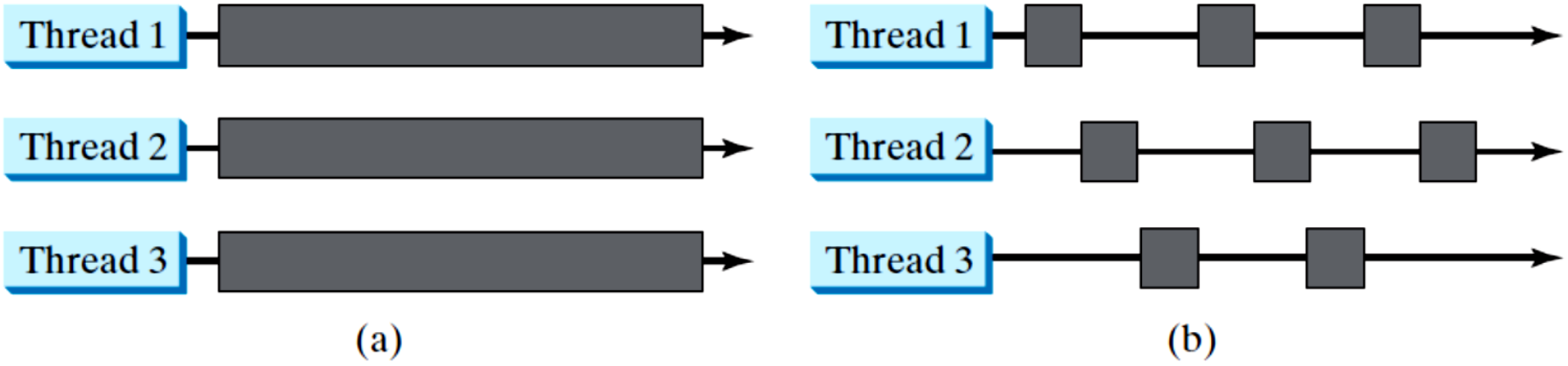
- *Multithreading enables multiple tasks in a program to be executed concurrently.*

Now we introduce the concepts of threads and how multithreading programs can be developed in Java.

Thread Concepts

- *A program may consist of many tasks that can run concurrently. A thread is the flow of execution, from beginning to end, of a task.*

A *thread* provides the mechanism for running a task. With Java, you can launch multiple threads from a program concurrently.



Creating Tasks and Threads

A task class must implement the **Runnable** *interface.*
A task must be run from a thread.

Tasks are objects. To create tasks, you have to first define a class for tasks, which implements the Runnable interface.

The **Runnable** interface is rather simple. All it contains is the **run** method. You need to implement this method to tell the system how your thread is going to run.

Template

1

```
// Custom task class
public class TaskClass implements Runnable {
  ...
  public TaskClass(...) {
    ...
  }

  // Implement the run meth
  public void run() {
    // Tell system how to r
    ...
  }
  ...
}
```

2

```
// Client class
public class Client {
  ...
  public void someMethod() {
    ...
    // Create an instance of TaskClass
    TaskClass task = new TaskClass(...);

    // Create a thread
    Thread thread = new Thread(task);

    // Start a thread
    thread.start();
    ...
  }
  ...
}
```

Once you have defined a **TaskClass**, you can create a task using its constructor. For example,

```
TaskClass task = new TaskClass(...);
```

A task must be executed in a thread. The **Thread** class contains the constructors for creating threads and many useful methods for controlling threads. To create a thread for a task, use

```
Thread thread = new Thread(task);
```

You can then invoke the **start()** method to tell the JVM that the thread is ready to run, as follows:

```
thread.start();
```

The JVM will execute the task by invoking the task's **run()** method.

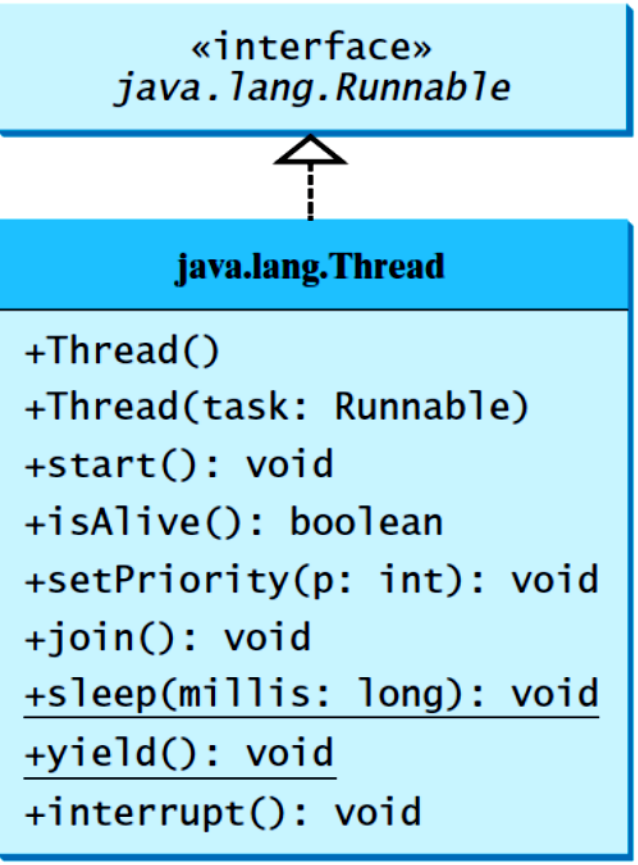
What is wrong here? How to correct?

```
public class Test implements Runnable{
        public static void main(String[] args) {
            new Test();
        }

        public Test() {
            // Test task = new Test();
            new Thread(this).start();
        }

        public void run() {
            System.out.println("test");
        }
}
```

What is wrong here (2)? How to correct?

```
public class Test implements Runnable{
        public static void main(String[] args) {
            new Test();
        }
        public Test() {
                Thread t = new Thread(this);
                t.start();
                //t.start();
        }
        public void run() {
            System.out.println("test");
        }
}
```

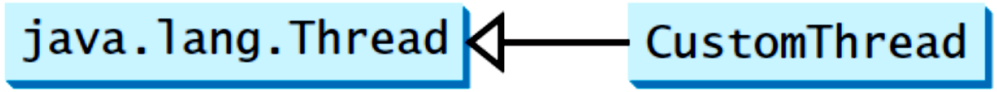
The Thread Class



Creates an empty thread.	
Creates a thread for a specified task.	
Starts the thread that causes the `run()` method to be invoked by the JVM.	
Tests whether the thread is currently running.	
Sets priority `p` (ranging from 1 to 10) for this thread.	
Waits for this thread to finish.	
Puts a thread to sleep for a specified time in milliseconds.	
Causes a thread to pause temporarily and allow other threads to execute.	
Interrupts this thread.	

Note

- Since the **Thread** class implements **Runnable**, you could define a class that extends **Thread** and implements the **run** method.
- Then create an object from the class and invoke its **start** method in a client program to start the thread.




```
// Custom thread class
public class CustomThread extends Thread {
  ...
  public CustomThr
    ...
  }


  // Override the
  public void run(
    // Tell system
    ...
  }
  ...
}
```

```
// Client class
public class Client {
  ...
  public void someMethod() {
    ...
    // Create a thread
    CustomThread thread1 = new CustomThread(...);

    // Start a thread
    thread1.start();
    ...

    // Create another thread
    CustomThread thread2 = new CustomThread(...);

    // Start a thread
    thread2.start();
  }
  ...
}
```

Case Study: Flashing Text

```
setLayout(new FlowLayout());
        label = new JLabel("Welcome!");
        label.setFont(new Font("",2,48));
        add(label);

        new Thread(new Runnable() {
            @Override
            public void run() {
                while(true){
                    try {
                        label.setFont(new Font("",2,48));
                        Thread.sleep(1000);
                        label.setFont(new Font("",2,8));
                        Thread.sleep(1000);
                    } catch (InterruptedException ex) {}
                }
            }

        }).start();
```

Back to Networking!

- Socket Server with multithreading.
- Example in netbeans.

Practice 27.04.18

- 1st Implement Server which receive message from Client, next capitalize it and send back.
- 2nd Implement 1st task but with multithreading

```
int port =8050;

        ServerSocket server;
        Socket socket;
        server = new ServerSocket(port);
        while(true) {
            socket = server.accept();
            Thread t = new Thread(new
 ThreadForClient(socket));
            t.start();

        }
```

```
• ThreadForClient(Socket socket) {
•       this.socket = socket;
•     }
•     @Override
•
•     public void run() {
•         try{
•             System.out.println("Connected");
•             in = new DataInputStream (socket.getInputStream());
•             out = new DataOutputStream (socket.getOutputStream());
•             System.out.println(message = in.readUTF());
•             out.writeUTF("I got "+ message);
•             socket.close();
•         } catch(Exception e){};
•     }
```